ANSIBLE – TASK - 03

1.Write a single ansible playbook which will install apache and nginx.

Note: Playbook should not be hardcoded and pass the variables from different file.

Ansible: Install Apache and Nginx Using a Single Variable-Driven Playbook

We will create one Ansible playbook to install both Apache and Nginx, while keeping all configuration variables in a separate file. This makes the playbook reusable and avoids hardcoding package names, service names, and installation settings.

1. Understand the architecture

Ansible Control Node

Your master / management server

SSH connection

Managed Nodes

Node 1

Apache

Node 2

Nginx

The same playbook installs both services on every targeted node, unless you choose separate host groups.

For this lab, we will install both Apache and Nginx on every managed node. We will use a separate YAML variables file, an inventory file, and a single main playbook.

2. Prerequisites

You need:

- One Ansible control node with Ansible installed.
- One or more Linux managed nodes reachable over SSH.

- SSH authentication configured between the control node and managed nodes.
- A user with sudo privileges on managed nodes.

Check Ansible on the control node:

ansible --version

Check connectivity after creating your inventory:

ansible all -i inventory.ini -m ping

A successful result should contain "ping": "pong" for each reachable host.

3. Create the project directory

On your Ansible control node, run:

mkdir -p ~/ansible-webserver-lab

cd ~/ansible-webserver-lab

Create these three files:

ansible-webserver-lab/

├── inventory.ini

├── vars.yml

└── install-webservers.yml

Each file has a separate responsibility:

File	Purpose
inventory.ini	Defines the managed servers
vars.yml	Stores package names, service names, and configuration
install-webservers.yml	Installs and configures both web servers

4. Configure the inventory file

Create the inventory file:

nano inventory.ini

Add your managed node's private IP address or DNS name:

[webservers]

node1 ansible_host=172.31.16.237 ansible_user=ec2-user

node2 ansible_host=172.31.18.138 ansible_user=ec2-user

Replace the example IP addresses with your actual managed-node IP addresses.

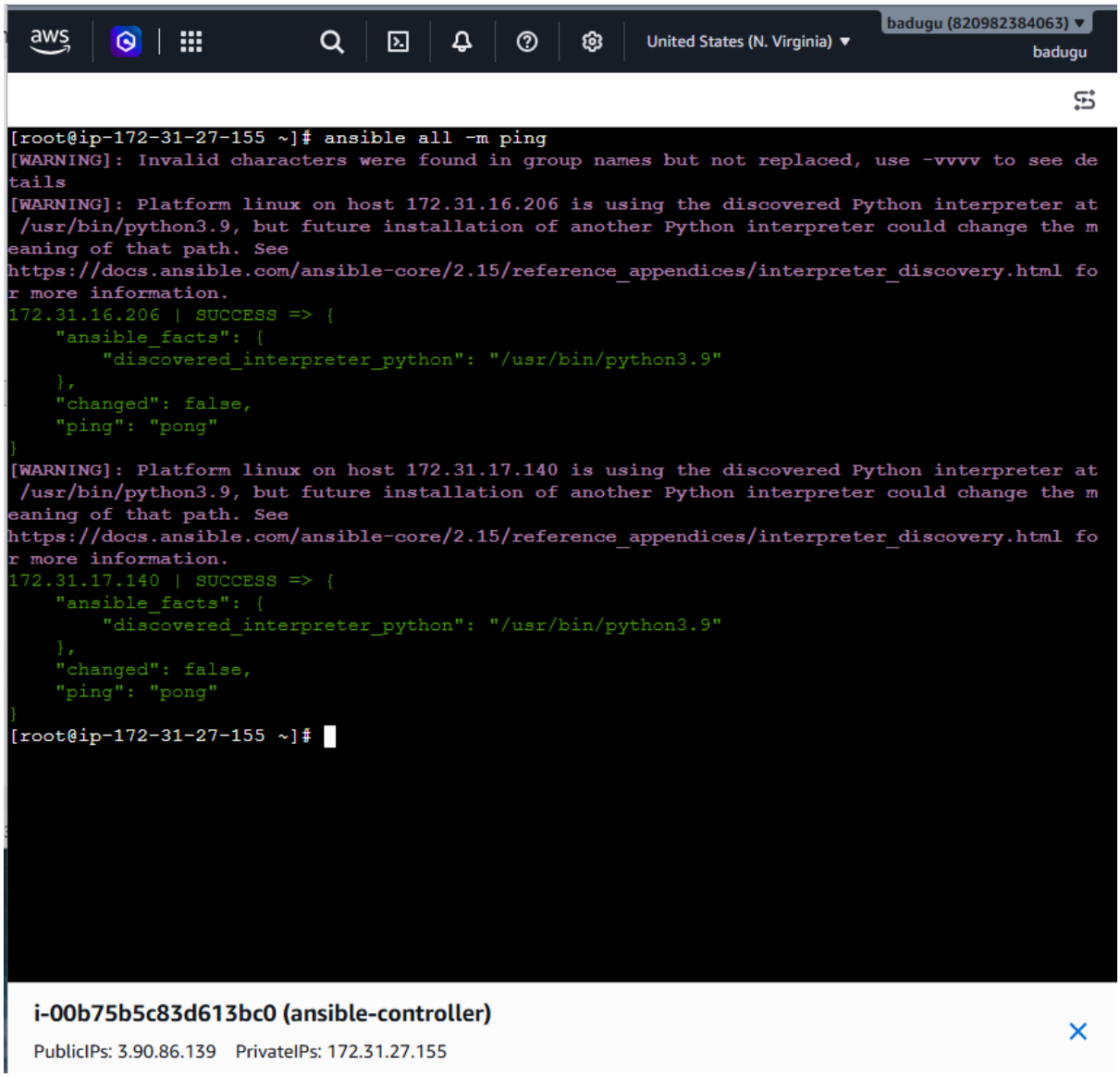
If you use Ubuntu, change ansible_user=ec2-user to ansible_user=ubuntu. If you use another Linux distribution, specify the appropriate SSH user.

Save and exit.

Test connectivity:

ansible -i inventory.ini webservers -m ping

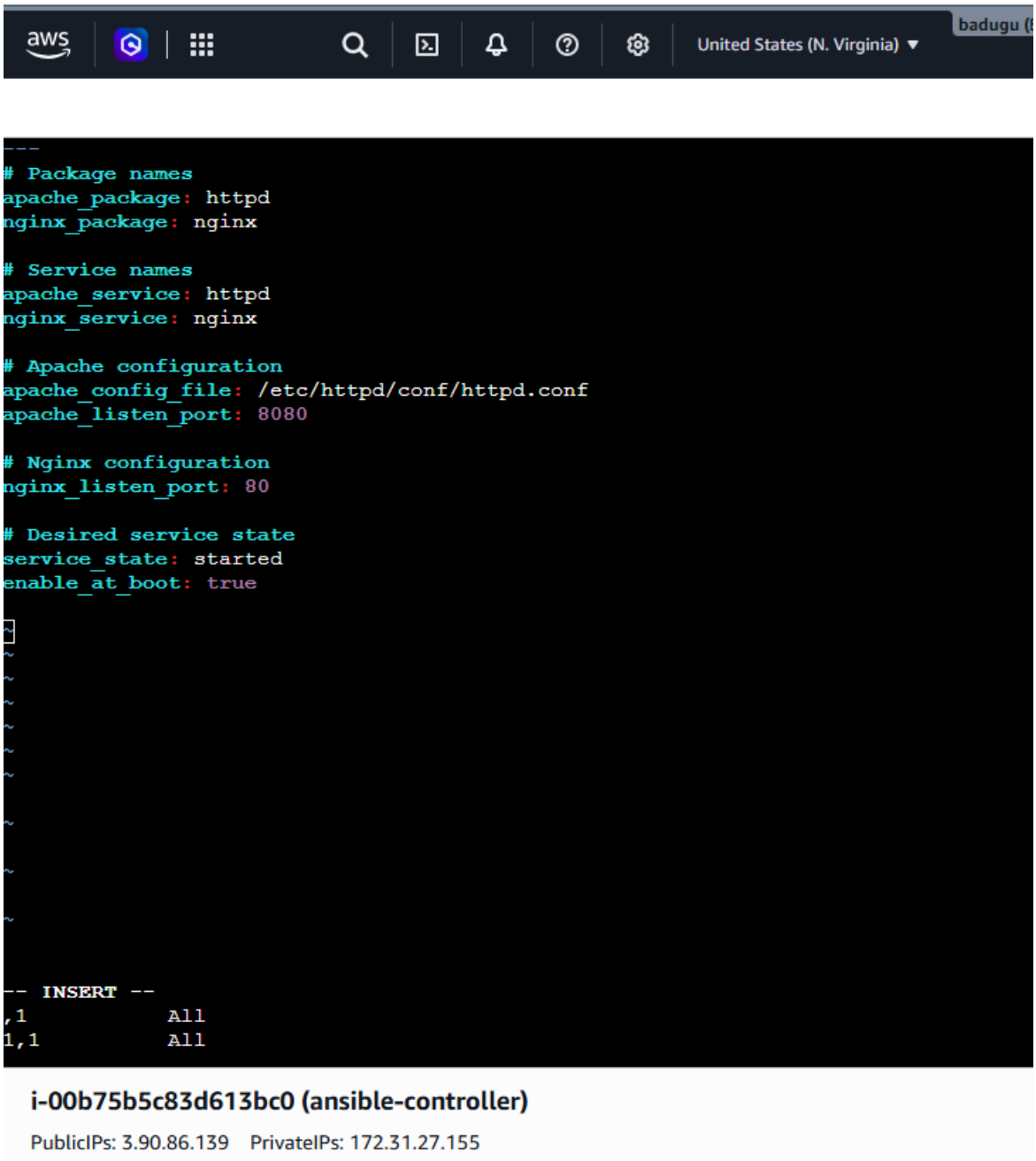
If your SSH user requires a private key, add the appropriate ansible_ssh_private_key_file setting to the inventory or supply it when running Ansible.



5. Create the separate variables file

Create the file:

nano vars.yml



Why are these variables important?

- apache_package and nginx_package specify which packages to install.
- apache_service and nginx_service specify which system services to manage.
- apache_listen_port sets Apache to port 8080.
- nginx_listen_port documents that Nginx will use port 80.
- service_state and enable_at_boot control service status and startup at boot.

These values target Amazon Linux / RHEL-family systems. Ubuntu and Debian use apache2 instead of httpd, with a different Apache configuration path. We will cover how to adapt the variables for those systems below.

6. Create the single Ansible playbook

Create the main playbook:

nano vars-main.yml




```yaml
---
- name: Install and configure Apache and Nginx
  hosts: all
  become: true

  vars_files:
    - vars.yml

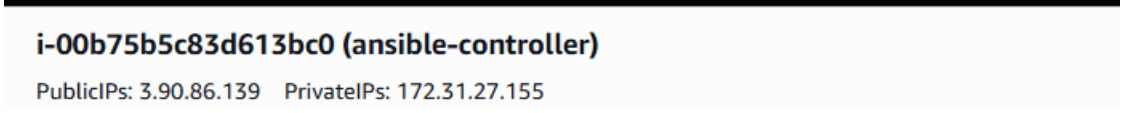
  tasks:
    - name: Install Apache and Nginx packages
      ansible.builtin.package:
        name:
          - "{{ apache_package }}"
          - "{{ nginx_package }}"
        state: present

    - name: Configure Apache listening port
      ansible.builtin.lineinfile:
        path: "{{ apache_config_file }}"
        regexp: '^Listen\s+'
        line: "Listen {{ apache_listen_port }}"
      notify: Restart Apache

    - name: Enable and start Apache
      ansible.builtin.service:
        name: "{{ apache_service }}"
        state: "{{ service_state }}"
        enabled: "{{ enable_at_boot }}"

    - name: Enable and start Nginx
      ansible.builtin.service:
        name: "{{ nginx_service }}"
        state: "{{ service_state }}"
        enabled: "{{ enable_at_boot }}"

  handlers:
    - name: Restart Apache
      ansible.builtin.service:
        name: "{{ apache_service }}"
```



Understand the important lines

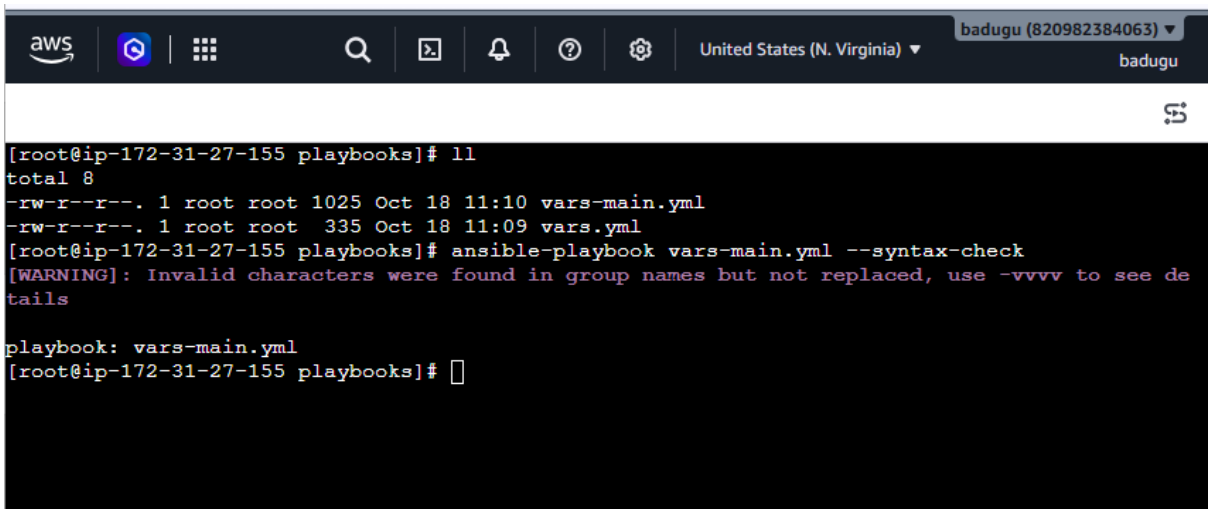
- hosts: webservers — runs against the hosts in the inventory group.
- become: true — uses privilege escalation for package installation and system configuration.
- vars_files — loads the variables from vars.yml.
- package — installs both packages without needing separate installation commands.
- lineinfile — changes Apache's listening port to 8080.
- service — starts both services and enables them at boot.
- handlers — restarts Apache only when its configuration changes.

Important: Apache and Nginx both commonly use port 80 by default. We configured Apache to use port 8080 so that both services can run on the same host without competing for the same listening port. Nginx remains on port 80.

7. Validate the playbook before execution

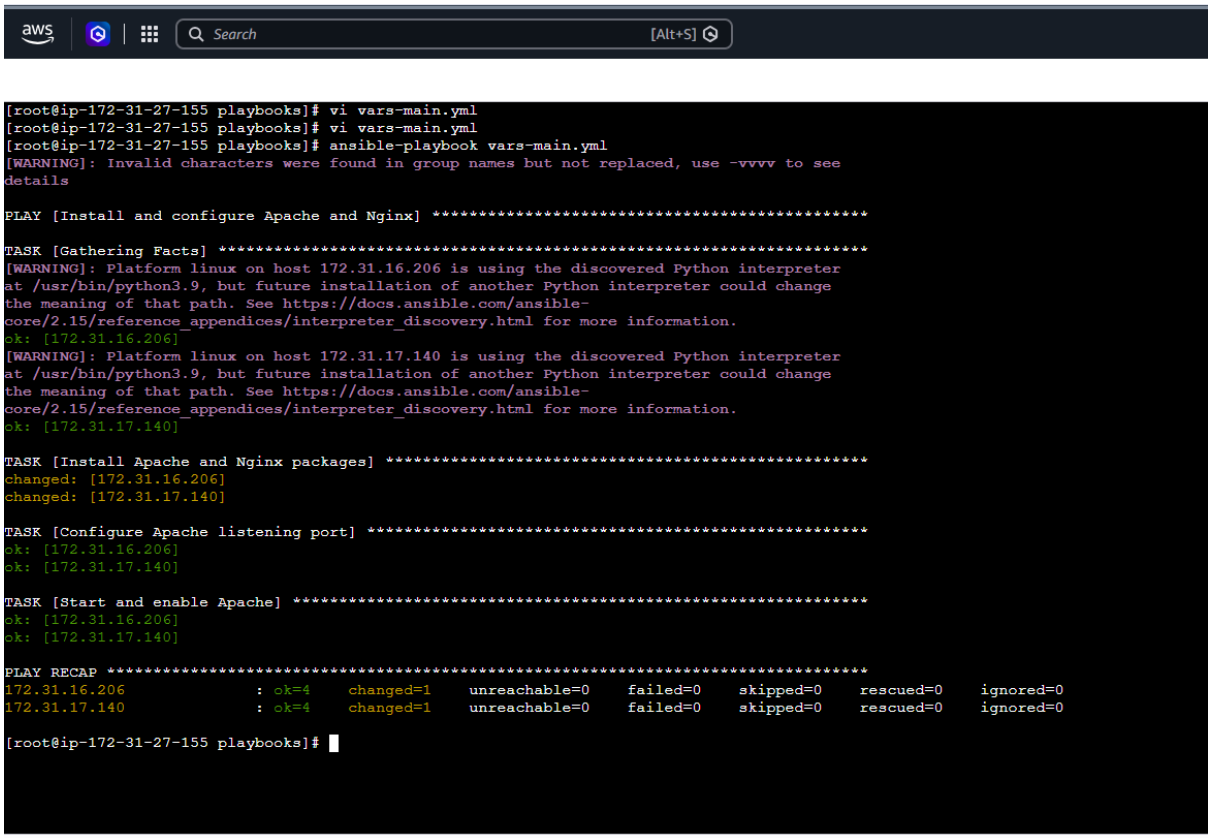
First, check that your files are present:

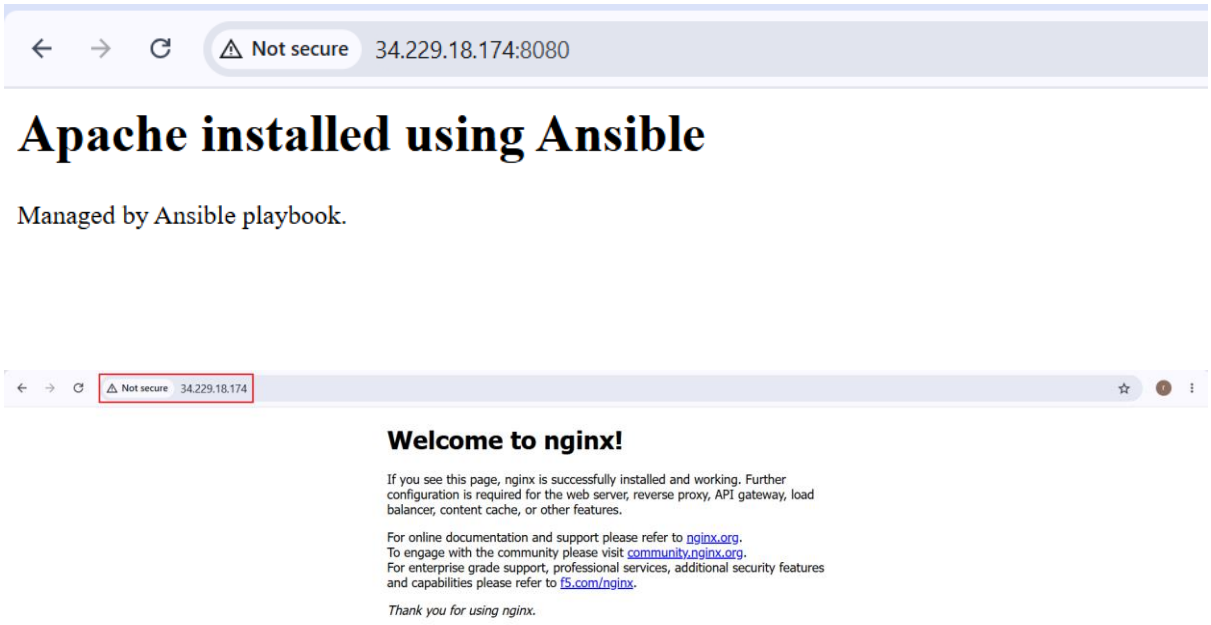
ls -l



8. Execute the playbook

Run the playbook:


aws
Search
[Alt+S]
[root@ip-172-31-27-155 playbooks]# vi vars-main.yml
[root@ip-172-31-27-155 playbooks]# vi vars-main.yml
[root@ip-172-31-27-155 playbooks]# ansible-playbook vars-main.yml
[WARNING]: Invalid characters were found in group names but not replaced, use -vvvv to see
details
PLAY [Install and configure Apache and Nginx] ***
TASK [Gathering Facts] ***
[WARNING]: Platform linux on host 172.31.16.206 is using the discovered Python interpreter
at /usr/bin/python3.9, but future installation of another Python interpreter could change
the meaning of that path. See https://docs.ansible.com/ansible-
core/2.15/reference_appendices/interpreter_discovery.html for more information.
ok: [172.31.16.206]
[WARNING]: Platform linux on host 172.31.17.140 is using the discovered Python interpreter
at /usr/bin/python3.9, but future installation of another Python interpreter could change
the meaning of that path. See https://docs.ansible.com/ansible-
core/2.15/reference_appendices/interpreter_discovery.html for more information.
ok: [172.31.17.140]
TASK [Install Apache and Nginx packages] ***
changed: [172.31.16.206]
changed: [172.31.17.140]
TASK [Configure Apache listening port] ***
ok: [172.31.16.206]
ok: [172.31.17.140]
TASK [Start and enable Apache] ***
ok: [172.31.16.206]
ok: [172.31.17.140]
PLAY RECAP ***
172.31.16.206 : ok=4 changed=1 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0
172.31.17.140 : ok=4 changed=1 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0
[root@ip-172-31-27-155 playbooks]#
i-00b75b5c83d613bc0 (ansible-controller)
PublicIPs: 3.90.86.139 PrivateIPs: 172.31.27.155

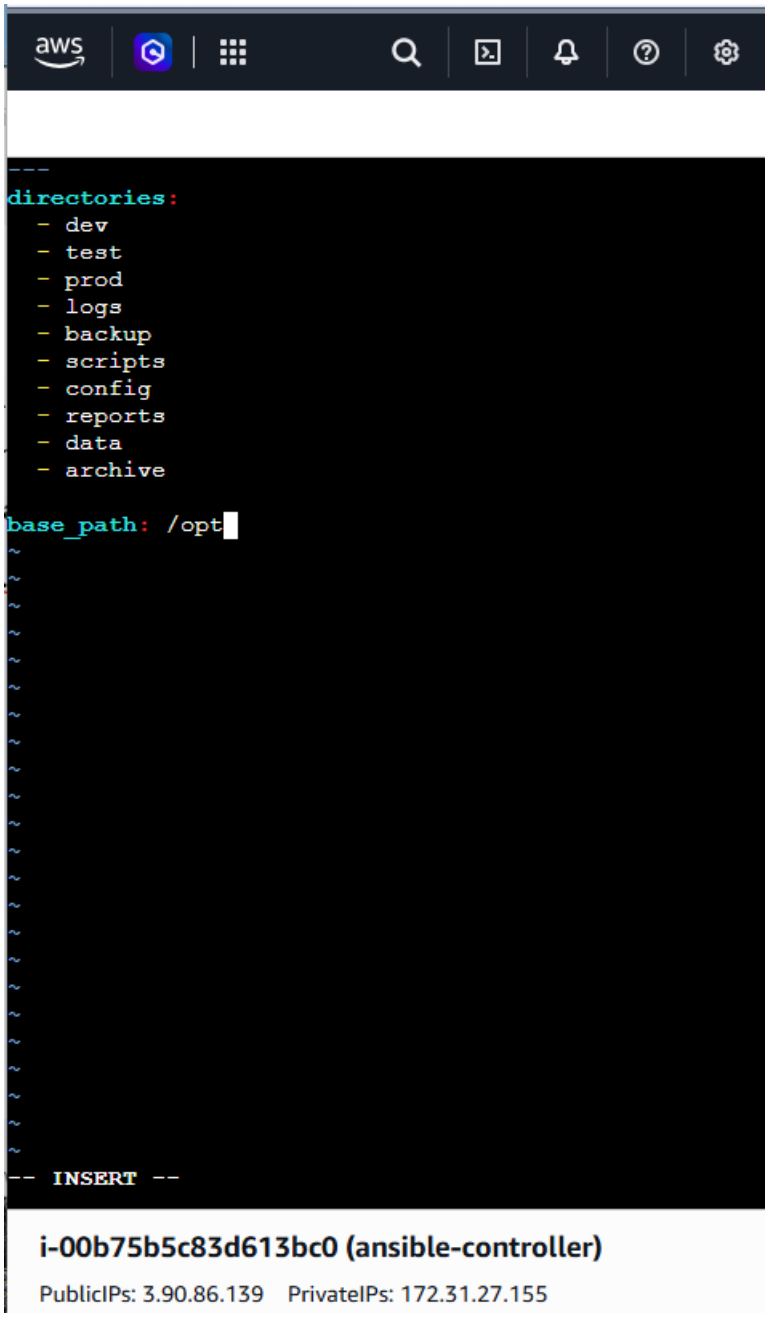

Not secure
34.229.18.174:8080
Apache installed using Ansible
Managed by Ansible playbook.

Not secure
34.229.18.174
Welcome to nginx!
If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.
For online documentation and support please refer to nginx.org.
To engage with the community please visit community.nginx.org.
For enterprise grade support, professional services, additional security features and capabilities please refer to f5.com/nginx.
Thank you for using nginx.

2.Ansible playbook to create 10 different directories with minimal code and directory names should be passed as variables.

Step 1: Create the variables file

Create vars.yml



Step 2: Create the playbook

Create vars_directories.yml



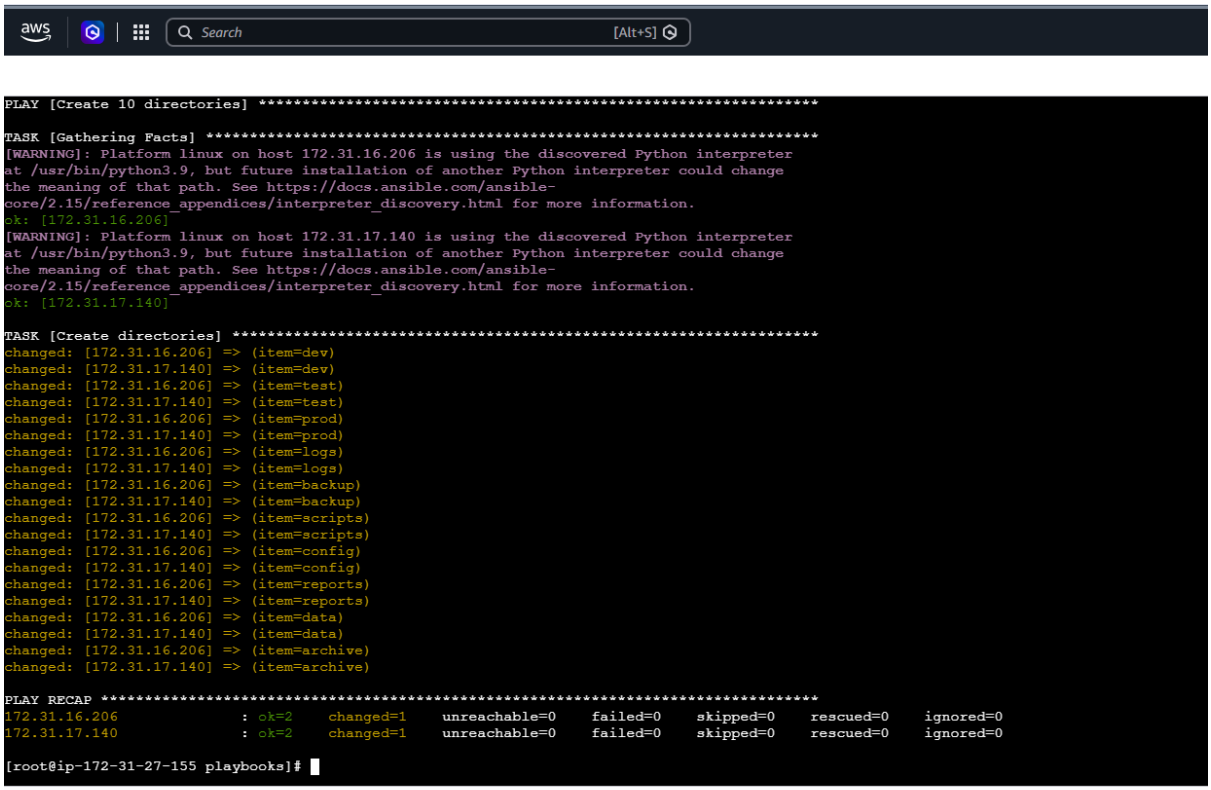

```yaml
---
- name: Create 10 directories
  hosts: all
  become: true

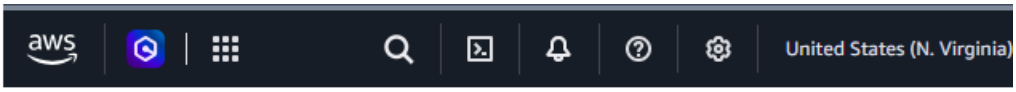
  vars_files:
    - vars_files.yml

  tasks:
    - name: Create directories
      ansible.builtin.file:
        path: "{{ base_path }}/{{ item }}"
        state: directory
        mode: '0755'
      loop: "{{ directories }}"
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
```

Step 3: Execute the playbook

ansible-playbook vars-directories.yml

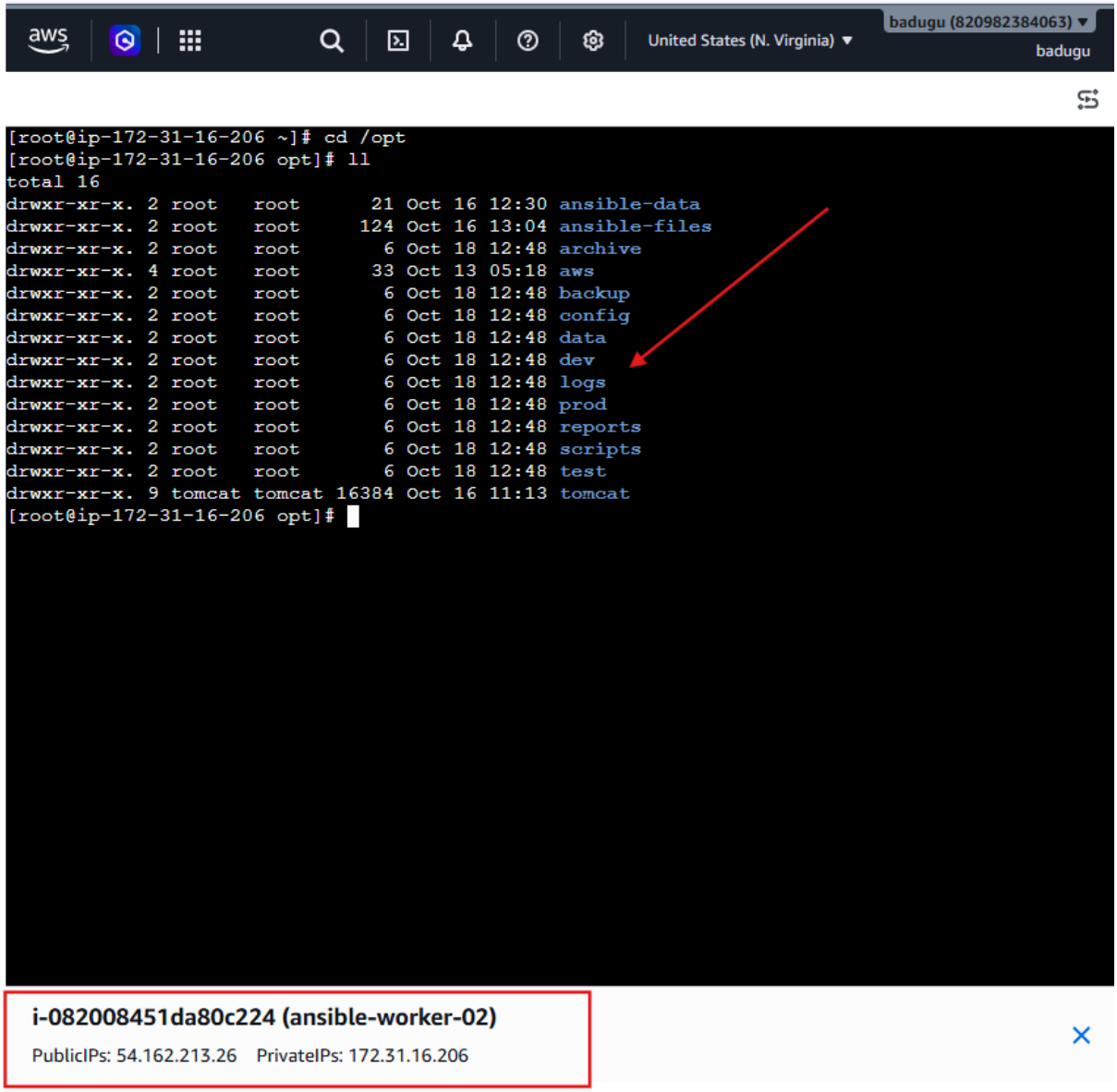

aws
Search
[Alt+S]
PLAY [Create 10 directories] ***
TASK [Gathering Facts] ***
[WARNING]: Platform linux on host 172.31.16.206 is using the discovered Python interpreter
at /usr/bin/python3.9, but future installation of another Python interpreter could change
the meaning of that path. See https://docs.ansible.com/ansible-
core/2.15/reference_appendices/interpreter_discovery.html for more information.
ok: [172.31.16.206]
[WARNING]: Platform linux on host 172.31.17.140 is using the discovered Python interpreter
at /usr/bin/python3.9, but future installation of another Python interpreter could change
the meaning of that path. See https://docs.ansible.com/ansible-
core/2.15/reference_appendices/interpreter_discovery.html for more information.
ok: [172.31.17.140]
TASK [Create directories] **
changed: [172.31.16.206] => (item=dev)
changed: [172.31.17.140] => (item=dev)
changed: [172.31.16.206] => (item=test)
changed: [172.31.17.140] => (item=test)
changed: [172.31.16.206] => (item=prod)
changed: [172.31.17.140] => (item=prod)
changed: [172.31.16.206] => (item=logs)
changed: [172.31.17.140] => (item=logs)
changed: [172.31.16.206] => (item=backup)
changed: [172.31.17.140] => (item=backup)
changed: [172.31.16.206] => (item=scripts)
changed: [172.31.17.140] => (item=scripts)
changed: [172.31.16.206] => (item=config)
changed: [172.31.17.140] => (item=config)
changed: [172.31.16.206] => (item=reports)
changed: [172.31.17.140] => (item=reports)
changed: [172.31.16.206] => (item=data)
changed: [172.31.17.140] => (item=data)
changed: [172.31.16.206] => (item=archive)
changed: [172.31.17.140] => (item=archive)
PLAY RECAP ***
172.31.16.206 : ok=2 changed=1 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0
172.31.17.140 : ok=2 changed=1 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0
[root@ip-172-31-27-155 playbooks]#
i-00b75b5c83d613bc0 (ansible-controller)
PublicIPs: 3.90.86.139 PrivateIPs: 172.31.27.155




```
[root@ip-172-31-17-140 ~]# cd /opt
[root@ip-172-31-17-140 opt]# ll
total 16
drwxr-xr-x. 2 root   root      21 Oct 16 12:30 ansible-data
drwxr-xr-x. 2 root   root     124 Oct 16 13:04 ansible-files
drwxr-xr-x. 2 root   root       6 Oct 18 12:48 archive
drwxr-xr-x. 4 root   root      33 Oct 13 05:18 aws
drwxr-xr-x. 2 root   root       6 Oct 18 12:48 backup
drwxr-xr-x. 2 root   root       6 Oct 18 12:48 config
drwxr-xr-x. 2 root   root       6 Oct 18 12:48 data
drwxr-xr-x. 2 root   root       6 Oct 18 12:48 dev
drwxr-xr-x. 2 root   root       6 Oct 18 12:48 logs
drwxr-xr-x. 2 root   root       6 Oct 18 12:48 prod
drwxr-xr-x. 2 root   root       6 Oct 18 12:48 reports
drwxr-xr-x. 2 root   root       6 Oct 18 12:48 scripts
drwxr-xr-x. 2 root   root       6 Oct 18 12:48 test
drwxr-xr-x. 9 tomcat tomcat 16384 Oct 16 11:13 tomcat
[root@ip-172-31-17-140 opt]#
```

i-01a6fcd85695419ff (ansible-worker-01)

PublicIPs: 34.229.18.174 PrivateIPs: 172.31.17.140


aws
United States (N. Virginia)
badugu (820982384063)
badugu
[root@ip-172-31-16-206 ~]# cd /opt
[root@ip-172-31-16-206 opt]# ll
total 16
drwxr-xr-x. 2 root root 21 Oct 16 12:30 ansible-data
drwxr-xr-x. 2 root root 124 Oct 16 13:04 ansible-files
drwxr-xr-x. 2 root root 6 Oct 18 12:48 archive
drwxr-xr-x. 4 root root 33 Oct 13 05:18 aws
drwxr-xr-x. 2 root root 6 Oct 18 12:48 backup
drwxr-xr-x. 2 root root 6 Oct 18 12:48 config
drwxr-xr-x. 2 root root 6 Oct 18 12:48 data
drwxr-xr-x. 2 root root 6 Oct 18 12:48 dev
drwxr-xr-x. 2 root root 6 Oct 18 12:48 logs
drwxr-xr-x. 2 root root 6 Oct 18 12:48 prod
drwxr-xr-x. 2 root root 6 Oct 18 12:48 reports
drwxr-xr-x. 2 root root 6 Oct 18 12:48 scripts
drwxr-xr-x. 2 root root 6 Oct 18 12:48 test
drwxr-xr-x. 9 tomcat tomcat 16384 Oct 16 11:13 tomcat
[root@ip-172-31-16-206 opt]#
i-082008451da80c224 (ansible-worker-02)
PublicIPs: 54.162.213.26 PrivateIPs: 172.31.16.206

3.Ansible playbook to copy ssh-keygen from master to worker nodes.

Note: a) Provision new 3 ec2 machines, one master and two worker nodes.

b) Create common user called ansadm and provide sudo priviliges on 3 ec2 instances. c) Create ssh-keygen in master and your playbook should copy the keygen making it password less authentication.

Configure Passwordless SSH from Master to Two Worker Nodes

In this lab, we will provision three EC2 instances, create a common Linux user named ansadm on all three machines, grant sudo privileges, generate an SSH key pair on the Ansible master, and use a single Ansible playbook to configure passwordless SSH authentication to both worker nodes.

1. Understand the architecture

EC2 Master — Ansible Control Node

User: ansadm

Generates SSH key pair

Runs the Ansible playbook

SSH public key authentication

Worker Node 1

User: ansadm

Authorized public key

Worker Node 2

User: ansadm

Authorized public key

What we will achieve

- Three EC2 instances: one master and two workers.
- A common ansadm user with sudo privileges on all three machines.
- An SSH key pair generated on the master.

- The master's public key added to both workers.
- Passwordless SSH from the master to each worker as ansadm.

Security rule: We will copy only the public key (id_rsa.pub) to the workers. The private key (id_rsa) must remain on the master and must never be distributed.

2. Provision three EC2 instances in AWS

Open the AWS Console and navigate to EC2 → Instances → Launch instances.

Launch three instances with these suggested settings:

Setting	**Master**	**Worker 1**	**Worker 2**
Name	ansible-master	ansible-worker1	ansible-worker2
OS	Amazon Linux 2023	Amazon Linux 2023	Amazon Linux 2023
Instance type	t3.micro	t3.micro	t3.micro
Network	Same VPC	Same VPC	Same VPC
SSH access	TCP 22	TCP 22	TCP 22

Use an appropriate EC2 key pair to access the instances initially. Choose an instance type and storage configuration compatible with your AWS account, quota, and current pricing.

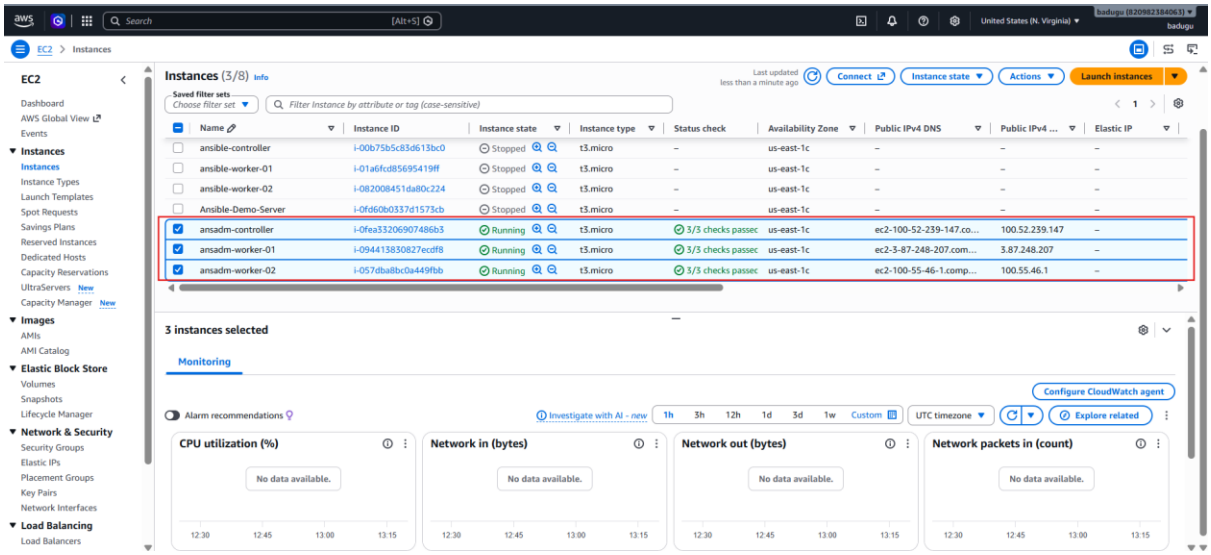
Configure the security group

Allow inbound SSH (TCP port 22) from:

- Your trusted public IP address to the master.
- The master’s private IP address or master security group to both workers.

Ensure the workers can reach the master and the master can reach the workers over the private network. Avoid opening SSH to 0.0.0.0/0.

Record each instance's private IP address and the master’s public IP address, if needed for your initial connection.



3. Connect to all three instances and create ansadm

Initially, connect using the EC2 default user and your downloaded EC2 private key.

For Amazon Linux 2023, the default user is usually ec2-user.

From your local terminal:

ssh -i your-ec2-key.pem ec2-user@MASTER_PUBLIC_IP

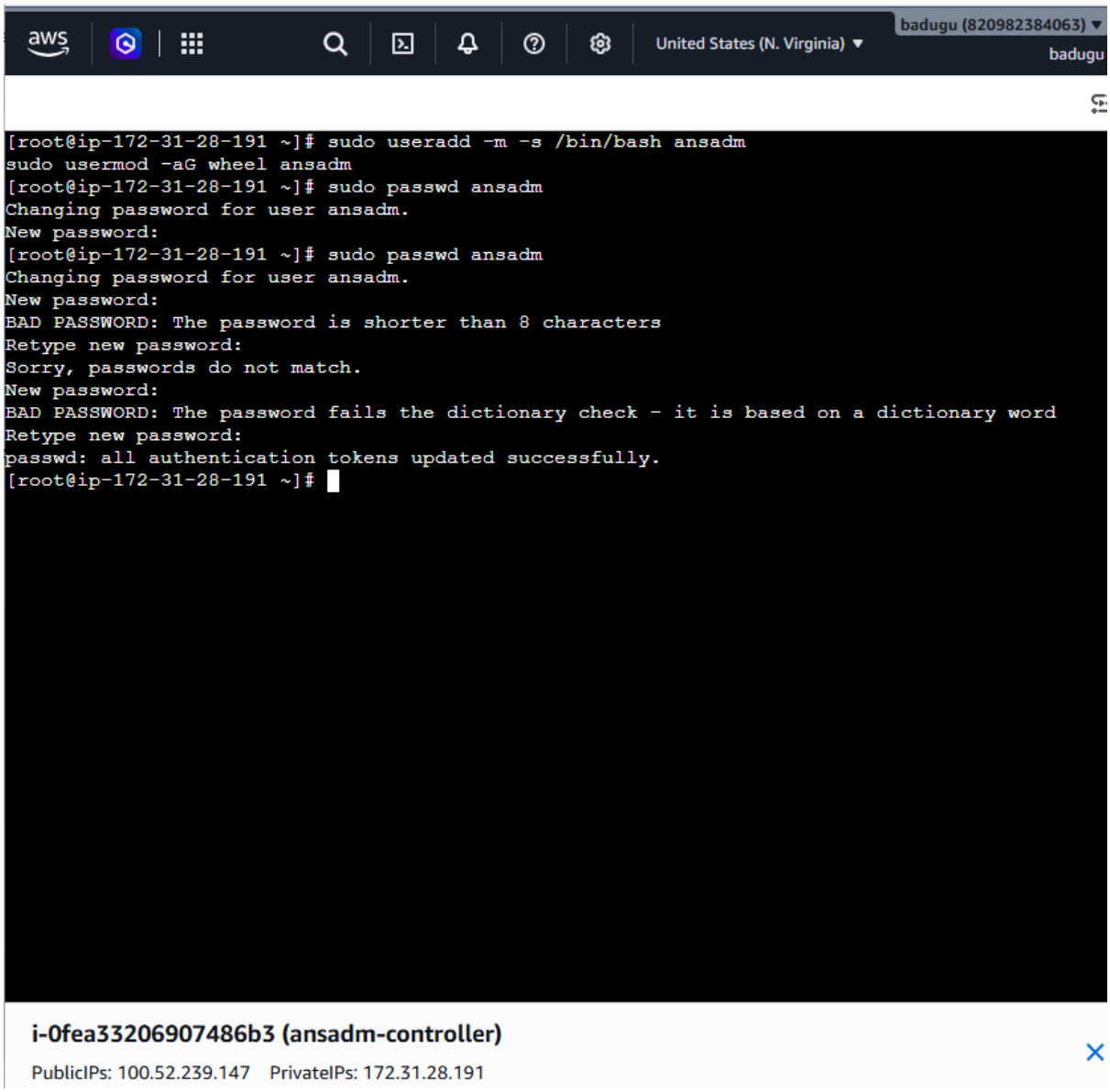
Run the following commands on each of the three EC2 instances:

sudo useradd -m -s /bin/bash ansadm

sudo usermod -aG wheel ansadm

Set a password if required by your organization's sudo policy:

sudo passwd ansadm



For this lab, we will configure passwordless sudo for ansadm, which is common for automation accounts. Only do this on machines where you administer access.

Create the sudoers configuration safely:

```
echo 'ansadm ALL=(ALL) NOPASSWD: ALL' | \
  sudo tee /etc/sudoers.d/ansadm
```

```
sudo chmod 440 /etc/sudoers.d/ansadm
```

```
sudo visudo -cf /etc/sudoers.d/ansadm
```

Expected validation:

/etc/sudoers.d/ansadm: parsed OK

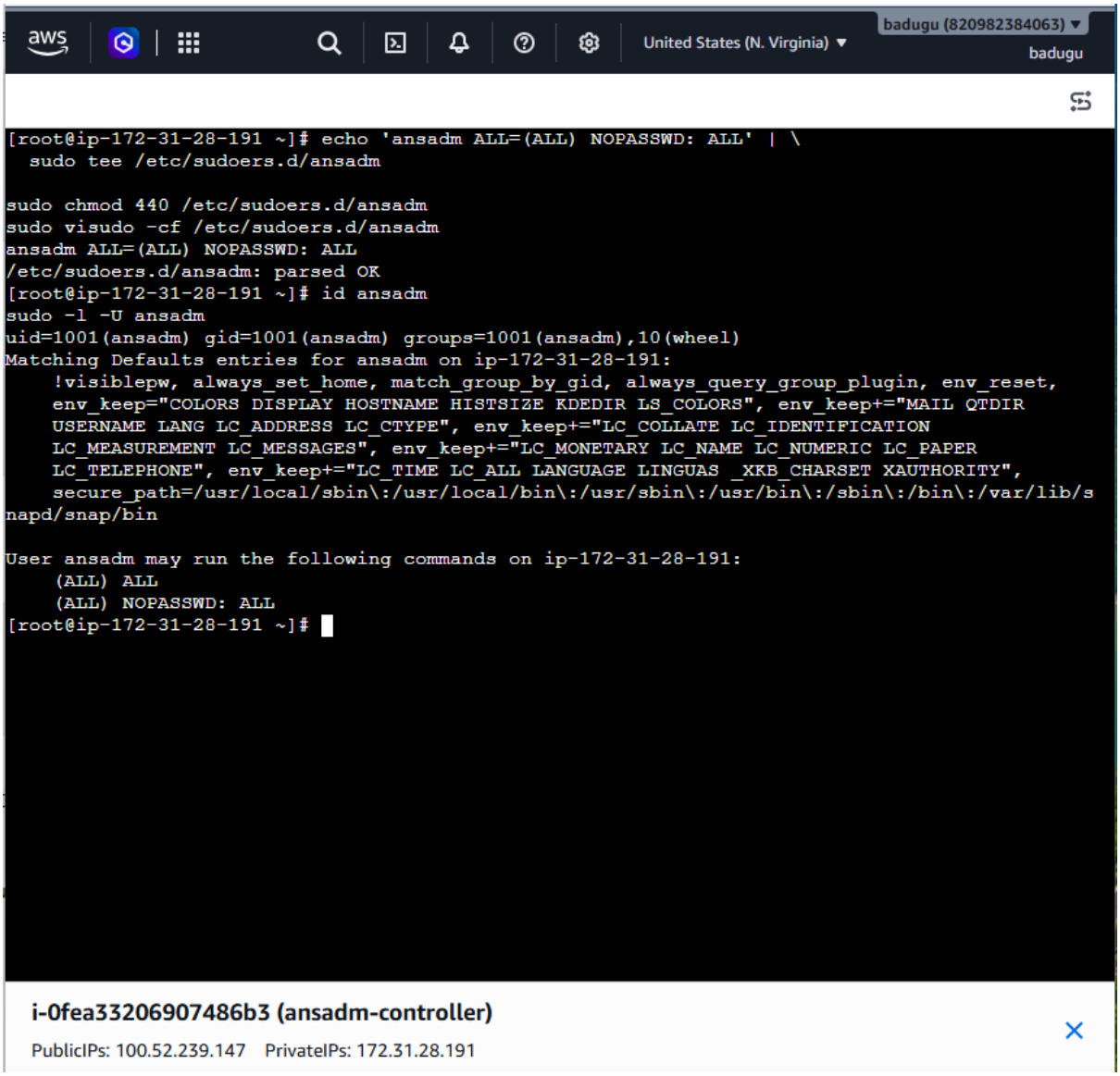
Verify the user:

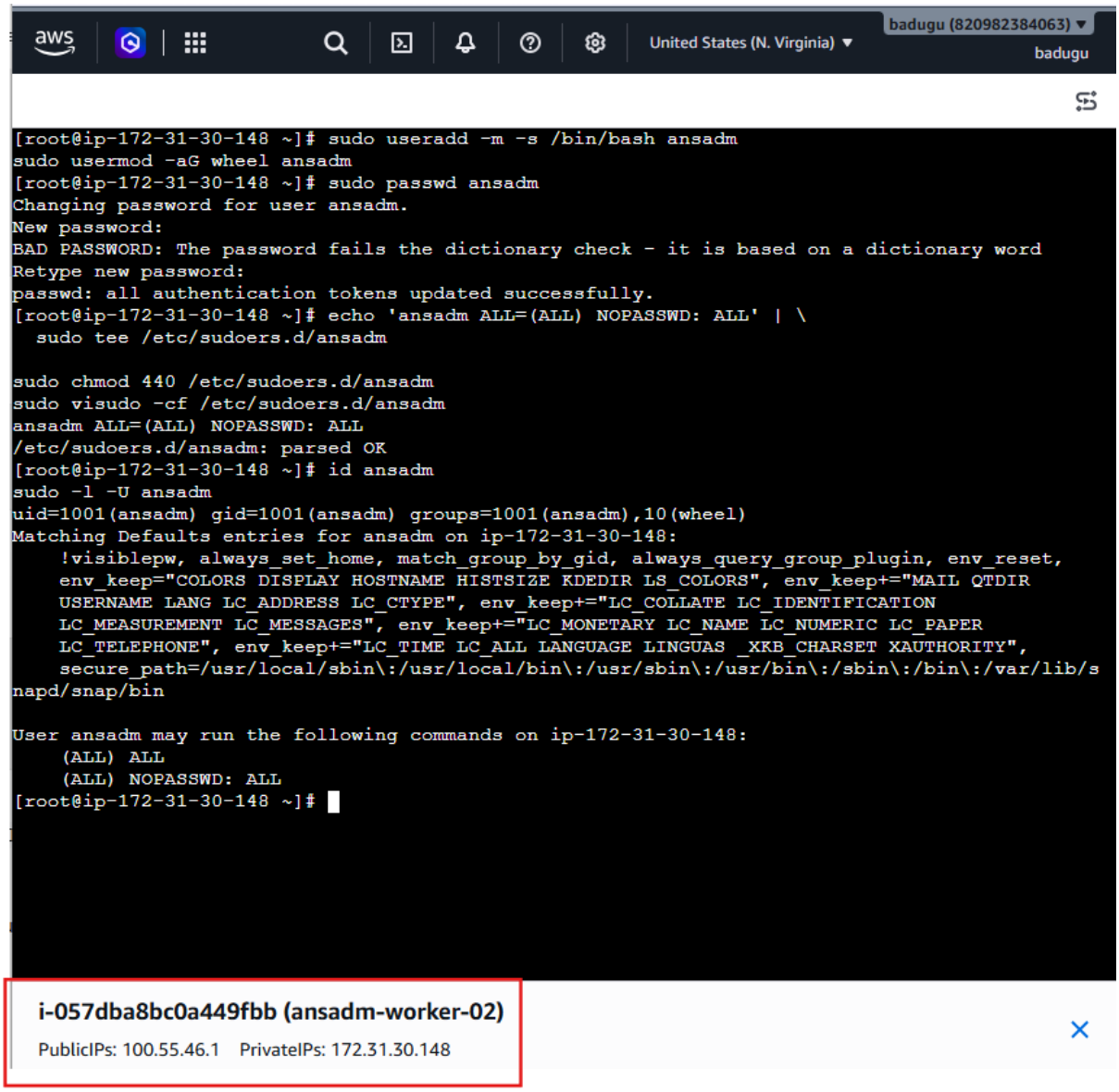
id ansadm

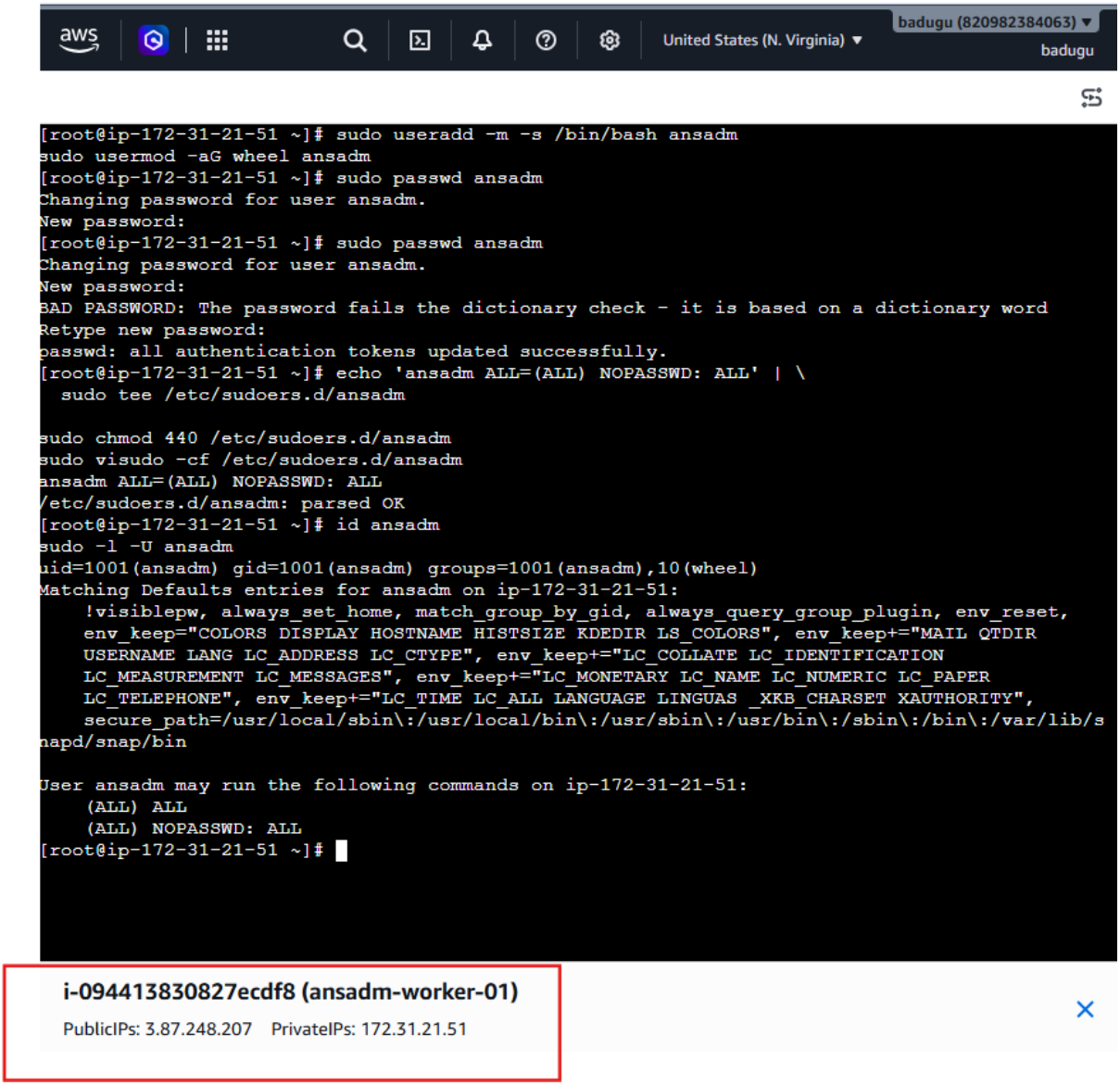
sudo -l -U ansadm

Repeat the steps on ansible-master, ansible-worker1, and ansible-worker2.

If ansadm already exists, don't run useradd again. Check the account with id ansadm and configure its permissions as needed.




aws
United States (N. Virginia)
badugu (820982384063)
badugu
[root@ip-172-31-30-148 ~]# sudo useradd -m -s /bin/bash ansadm
sudo usermod -aG wheel ansadm
[root@ip-172-31-30-148 ~]# sudo passwd ansadm
Changing password for user ansadm.
New password:
BAD PASSWORD: The password fails the dictionary check - it is based on a dictionary word
Retype new password:
passwd: all authentication tokens updated successfully.
[root@ip-172-31-30-148 ~]# echo 'ansadm ALL=(ALL) NOPASSWD: ALL' | \
sudo tee /etc/sudoers.d/ansadm
sudo chmod 440 /etc/sudoers.d/ansadm
sudo visudo -cf /etc/sudoers.d/ansadm
ansadm ALL=(ALL) NOPASSWD: ALL
/etc/sudoers.d/ansadm: parsed OK
[root@ip-172-31-30-148 ~]# id ansadm
sudo -l -U ansadm
uid=1001(ansadm) gid=1001(ansadm) groups=1001(ansadm),10(wheel)
Matching Defaults entries for ansadm on ip-172-31-30-148:
!visiblepw, always_set_home, match_group_by_gid, always_query_group_plugin, env_reset,
env_keep="COLORS DISPLAY HOSTNAME HISTSIZE KDEDIR LS_COLORS", env_keep+="MAIL QTDIR
USERNAME LANG LC_ADDRESS LC_CTYPE", env_keep+="LC_COLLATE LC_IDENTIFICATION
LC_MEASUREMENT LC_MESSAGES", env_keep+="LC_MONETARY LC_NAME LC_NUMERIC LC_PAPER
LC_TELEPHONE", env_keep+="LC_TIME LC_ALL LANGUAGE LINGUAS _XKB_CHARSET XAUTHORITY",
secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/var/lib/snapd/snap/bin
User ansadm may run the following commands on ip-172-31-30-148:
(ALL) ALL
(ALL) NOPASSWD: ALL
[root@ip-172-31-30-148 ~]#
i-057dba8bc0a449fbb (ansadm-worker-02)
PublicIPs: 100.55.46.1 PrivateIPs: 172.31.30.148



4. Install Ansible on the master

Connect to the master and install Ansible.

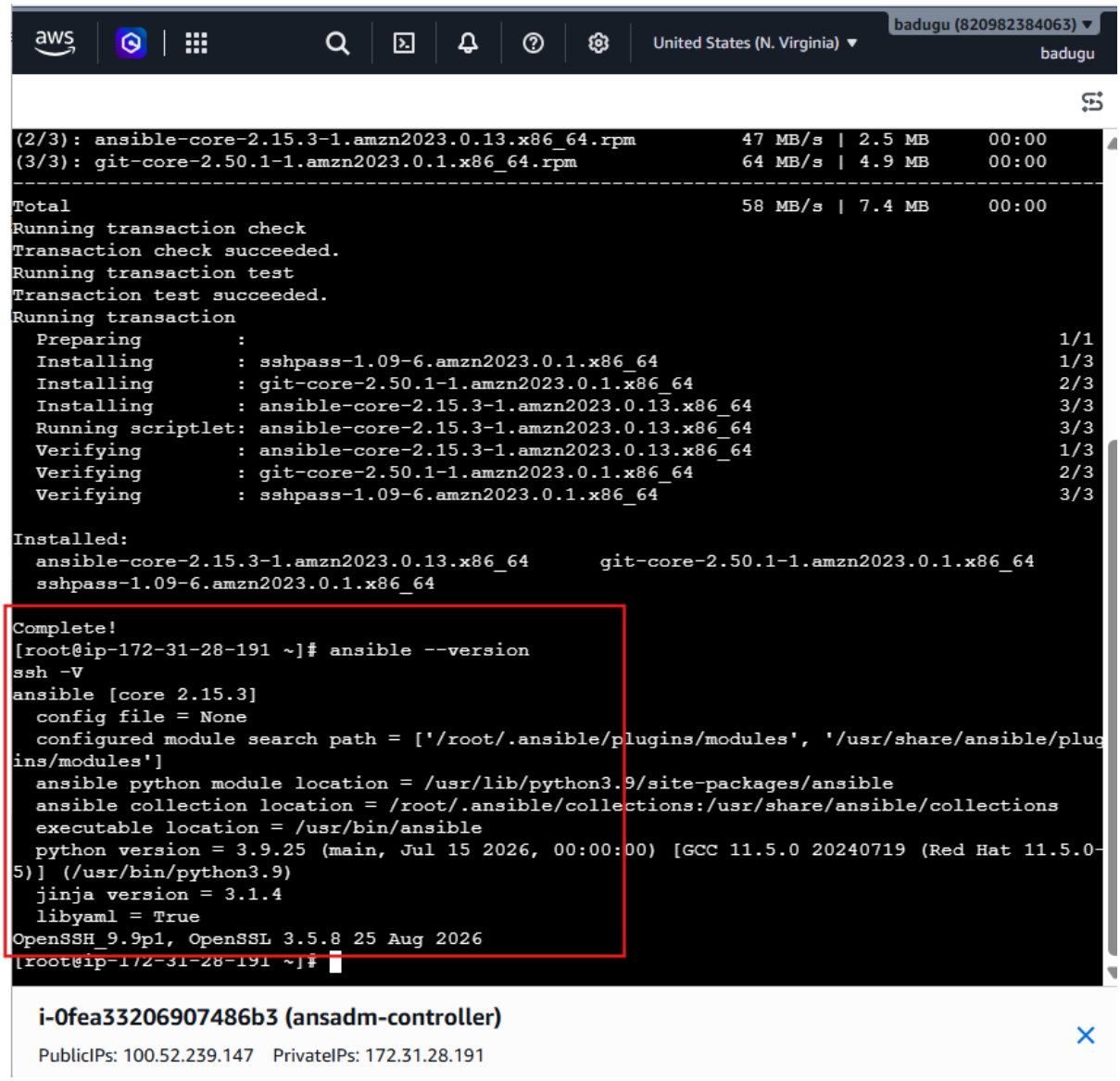
For Amazon Linux 2023, try:

sudo dnf install -y ansible-core openssh-clients

Verify:

ansible --version

ssh -V



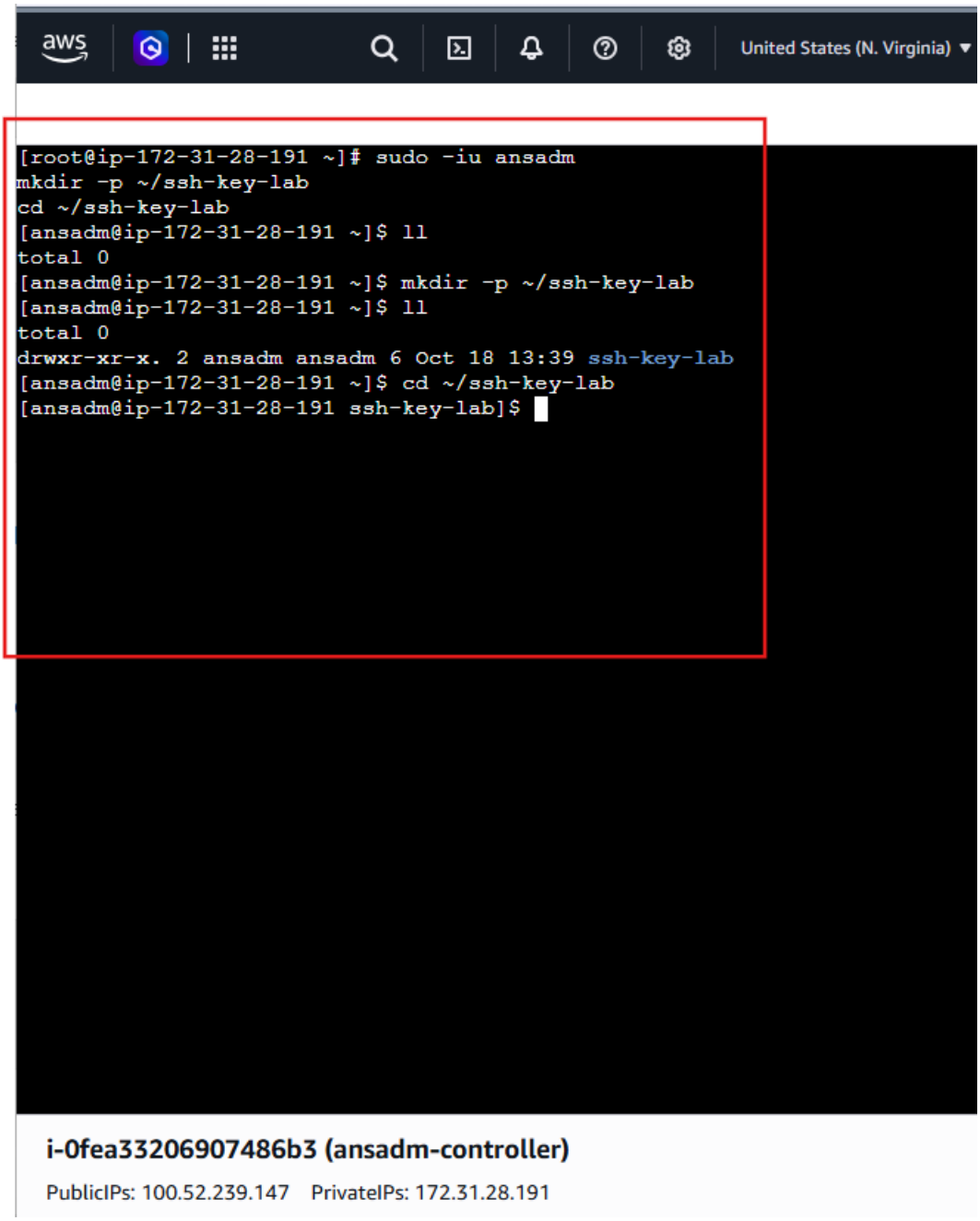
Create the project directory:

sudo -iu ansadm

mkdir -p ~/ssh-key-lab

cd ~/ssh-key-lab

The remaining commands in this section should be run as ansadm unless stated otherwise.



5. Make the initial EC2 SSH key available to ansadm

Ansible needs the original EC2 private key to connect to the workers before their ansadm accounts have been configured for passwordless SSH.

Copy your downloaded EC2 .pem file to the master using a secure method. For example, from your local terminal:

scp -i your-ec2-key.pem your-ec2-key.pem \

ec2-user@MASTER_PUBLIC_IP:/tmp/ec2-access.pem

Then, on the master, move it to the ansadm home directory:

```
sudo install -o ansadm -g ansadm -m 600 \
  /tmp/ec2-access.pem /home/ansadm/ec2-access.pem


sudo rm -f /tmp/ec2-access.pem
```

Check permissions:

```
ls -l /home/ansadm/ec2-access.pem
```

Expected permissions should resemble:

```
-rw------- 1 ansadm ansadm ... /home/ansadm/ec2-access.pem
```

This EC2 key is used only for the initial connection. Once key-based authentication is working, you can remove the copy if it is no longer required.

6. Generate the SSH key pair on the master

Switch to ansadm if necessary:

```
sudo -iu ansadm
```

Generate the key pair:

```
ssh-keygen -t ed25519 -C "ansadm@ansible-master"
```

When prompted:

Enter file in which to save the key:

Press Enter to accept:

```
/home/ansadm/.ssh/id_ed25519
```

For this lab, you can leave the passphrase empty to enable unattended SSH. In production, consider using an SSH agent, a protected key, or an approved secrets-management solution.

Verify that both files exist:

```
ls -l ~/.ssh/
```

You should see:

```
id_ed25519

id_ed25519.pub
```

- id_ed25519 — private key; keep it on the master.
- id_ed25519.pub — public key; this is what we will install on both workers.

Display the public key if you want to inspect it:

cat ~/.ssh/id_ed25519.pub

You do not need to manually copy and paste this key. The playbook will read it from the master.

7. Create the Ansible inventory

From /home/ansadm/ssh-key-lab, create inventory.ini:

nano inventory.ini

Add the private IP addresses of your two workers:

[workers]

worker1 ansible_host=172.31.18.138

worker2 ansible_host=172.31.16.237

[workers:vars]

ansible_user=ec2-user

ansible_ssh_private_key_file=/home/ansadm/ec2-access.pem

ansible_ssh_common_args='-o IdentitiesOnly=yes'

Replace the example IP addresses with the actual private IPs of your workers. Ensure they are reachable from the master over SSH.

Here we deliberately use ec2-user for the initial Ansible connection, because the workers do not yet have the master's new public key installed. The playbook will configure ansadm on the workers.

Test the initial connection:

ansible -i inventory.ini workers -m ping

Expected result: both workers respond with pong.

If the connection fails, check the IP addresses, security group rules, private key permissions, and SSH connectivity before proceeding.

8. Write the single Ansible playbook

Create the playbook:

nano copy-ssh-key.yml

Paste this content:

```
---
- name: Configure passwordless SSH for ansadm
  hosts: workers
  become: true
  gather_facts: false

  vars:
    ansadm_user: ansadm
    master_public_key: "{{ lookup('file', '/home/ansadm/.ssh/id_ed25519.pub') }}"

  tasks:
    - name: Ensure ansadm exists on worker nodes
      ansible.builtin.user:
        name: "{{ ansadm_user }}"
        state: present
        create_home: true
        shell: /bin/bash

    - name: Configure passwordless sudo for ansadm
```

```
    ansible.builtin.copy:
      dest: /etc/sudoers.d/ansadm
      content: "{{ ansadm_user }} ALL=(ALL) NOPASSWD: ALL\n"
      owner: root
      group: root
      mode: '0440'
      validate: '/usr/sbin/visudo -cf %s'

  - name: Install master's public key for ansadm
    ansible.posix.authorized_key:
      user: "{{ ansadm_user }}"
      key: "{{ master_public_key }}"
      state: present
      manage_dir: true
```

How this playbook works

1. Ensures that the ansadm account exists on both workers.
2. Configures sudo privileges for automation.
3. Reads the public key from the master using Ansible's lookup('file', ...).
4. Adds the public key to each worker's /home/ansadm/.ssh/authorized_keys.

The playbook does not transfer the private key. The authorized_key module installs the public key on the destination machines.

9. Install the required Ansible collection

The ansible.posix.authorized_key module belongs to the ansible.posix collection. Install it on the master as ansadm:

ansible-galaxy collection install ansible.posix

Verify it is available:

ansible-doc ansible.posix.authorized_key

10. Validate and run the playbook

First, check the project files:

cd ~/ssh-key-lab

ls -l

You should have:

inventory.ini

copy-ssh-key.yml

Validate the YAML:

ansible-playbook -i inventory.ini copy-ssh-key.yml --syntax-check

Run the playbook:

ansible-playbook -i inventory.ini copy-ssh-key.yml

If everything succeeds, the recap should show both workers with failed=0 and unreachable=0.

You can run the playbook again safely. The user, sudoers file, and authorized key tasks are designed to be idempotent.

11. Verify passwordless SSH authentication

After the playbook succeeds, connect from the master as ansadm and test each worker.

ssh -i ~/.ssh/id_ed25519 ansadm@172.31.18.138

Then test the second worker:

ssh -i ~/.ssh/id_ed25519 ansadm@172.31.16.237

Replace the IPs with your worker private IPs. Because you generated the key without a passphrase, SSH should connect without asking for the ansadm password.

On each worker, verify:

whoami

sudo whoami

Expected output:

ansadm

root

Exit back to the master:

exit

12. Update the inventory to use ansadm

Now that public-key authentication works, update inventory.ini so Ansible uses ansadm rather than ec2-user.

[workers]

worker1 ansible_host=172.31.18.138

worker2 ansible_host=172.31.16.237

[workers:vars]

ansible_user=ansadm

ansible_ssh_private_key_file=/home/ansadm/.ssh/id_ed25519

ansible_ssh_common_args='-o IdentitiesOnly=yes'

Test Ansible connectivity:

ansible -i inventory.ini workers -m ping

Expected result:

```
worker1 | SUCCESS => {
    "changed": false,
    "ping": "pong"
}

worker2 | SUCCESS => {
    "changed": false,
    "ping": "pong"
```

}

Verify sudo access through Ansible:

ansible -i inventory.ini workers -b -m command -a "whoami"

Both workers should return:

root

13. Troubleshooting common errors

Error	What to check
Permission denied (publickey)	Verify the EC2 access key for initial access and authorized_keys for ansadm access.
UNREACHABLE	Check worker private IPs, routing, security groups, and SSH connectivity.
Missing sudo password	Verify sudoers configuration and NOPASSWD policy.
Could not find or access ... id_ed25519.pub	Verify the public key exists at /home/ansadm/.ssh/id_ed25519.pub on the control node.
ansible.posix.authorized_key not found	Install the ansible.posix collection.
SSH still requests a password	Check ~/.ssh ownership, authorized_keys permissions, and SSH server logs on the worker.

4.Ansible playbook to inject ansible vault variables.

Ansible Playbook to Inject Ansible Vault Variables

In this lab, we will store sensitive information such as database usernames and passwords in an Ansible Vault-encrypted variables file, then inject those variables into a playbook at runtime.

The main advantage is that you can keep passwords encrypted instead of writing them directly in your playbook.

1. Understand the architecture

vault.yml

Encrypted variables and secrets

install-app.yml

Loads secrets using vars_files

Managed EC2 / Linux server

Receives the required configuration securely

We will create three files:

ansible-vault-lab/

├── inventory.ini

├── vault.yml

└── install-app.yml

- inventory.ini — defines the target server.
- vault.yml — contains encrypted variables.
- install-app.yml — injects the variables into the task.

2. Prerequisites

On your Ansible control node, check that Ansible is installed:

ansible --version

Create the project directory:

mkdir -p ~/ansible-vault-lab

cd ~/ansible-vault-lab

3. Create the inventory file

Create inventory.ini:

nano inventory.ini

Add your managed server:

[appservers]

worker1 ansible_host=172.31.18.138 ansible_user=ec2-user

Replace the example IP with your worker's private IP. Ensure SSH access is configured and the control node can reach the worker.

If you're using your existing ansadm setup, you can instead use ansible_user=ansadm and the appropriate SSH private-key setting.

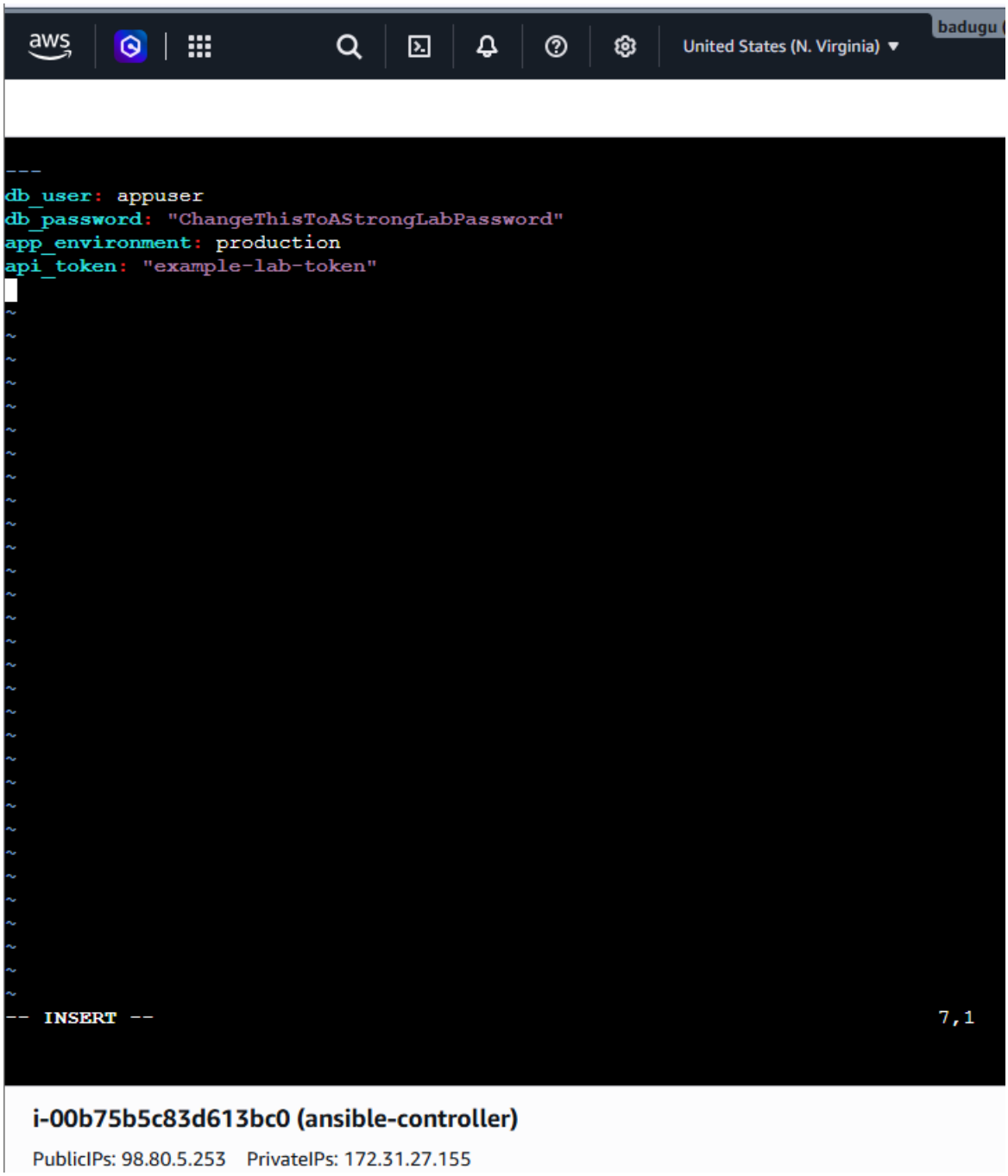
4. Create the Ansible Vault variables file

Run:

ansible-vault create vault.yml

Ansible will ask you to enter and confirm a Vault password. Choose a strong password and keep it safe.

An editor will open. Add the following YAML variables:



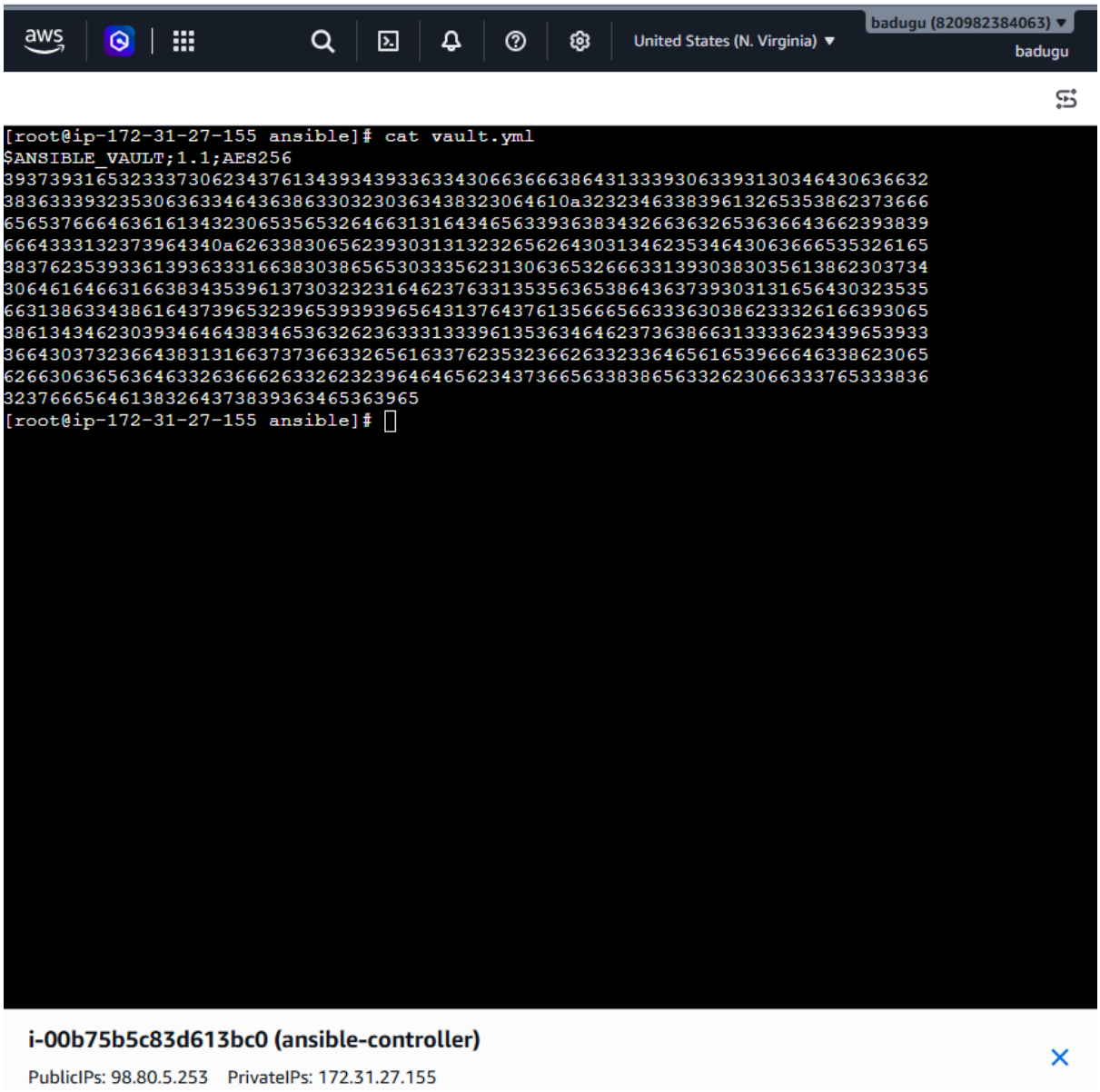
Important: These values are examples for learning. Replace them with appropriate values for your environment. Do not use real production secrets in a practice lab.

Important: These values are examples for learning. Replace them with appropriate values for your environment. Do not use real production secrets in a practice lab.

Ansible encrypts the file when you save it. Verify this with:

cat vault.yml

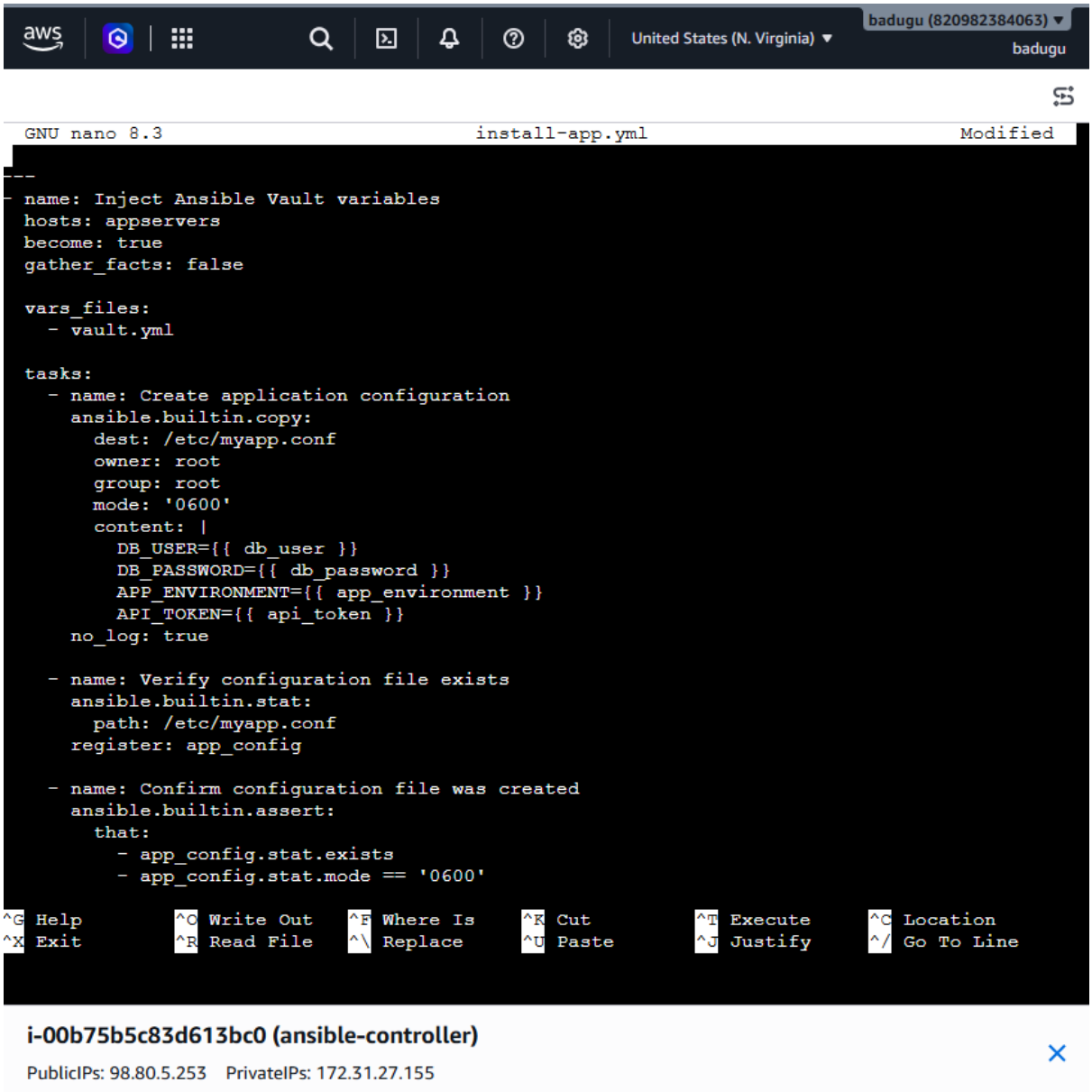
The contents should begin with something similar to:



5. Create the playbook to inject Vault variables

Create install-app.yml:

nano install-app.yml



How the playbook works

- vars_files: - vault.yml loads the encrypted variables into the play.
- db_user, db_password, app_environment, and api_token are substituted when the task runs.
- copy writes the resulting configuration to /etc/myapp.conf.
- mode: '0600' restricts the file to its owner, root.
- no_log: true suppresses task details that could expose secrets in Ansible output.
- stat and assert verify the configuration file exists and has the intended permissions.

This example writes secrets to a file on the managed server, so the destination must be protected and the application should genuinely need those values there. For production, use your organization's approved secret-delivery mechanism and avoid exposing secrets in logs, source control, or unnecessary files.

6. Validate the playbook

Check the project files:

ls -l

Expected:

inventory.ini

vault.yml

install-app.yml

Validate the playbook syntax and decrypt the Vault file when prompted:

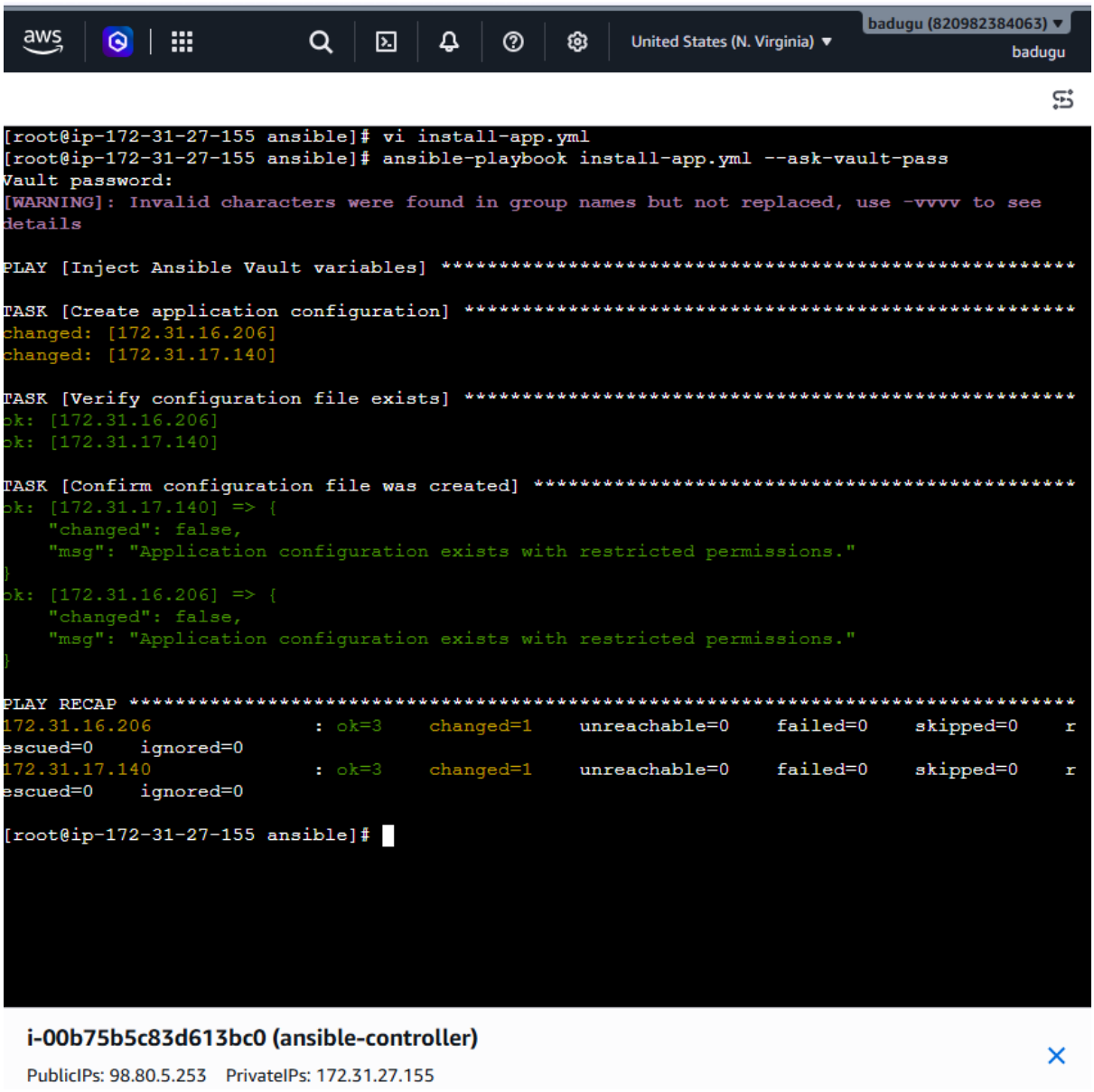
ansible-playbook -i inventory.ini install-app.yml --syntax-check --ask-vault-pass

Enter your Vault password when requested.

7. Execute the playbook

Run:

Enter the Vault password. Ansible decrypts the variables in memory as needed and uses them during execution.


aws
United States (N. Virginia)
badugu (820982384063)
badugu
[root@ip-172-31-27-155 ansible]# vi install-app.yml
[root@ip-172-31-27-155 ansible]# ansible-playbook install-app.yml --ask-vault-pass
Vault password:
[WARNING]: Invalid characters were found in group names but not replaced, use -vvvv to see details
PLAY [Inject Ansible Vault variables] ***
TASK [Create application configuration] ***
changed: [172.31.16.206]
changed: [172.31.17.140]
TASK [Verify configuration file exists] ***
ok: [172.31.16.206]
ok: [172.31.17.140]
TASK [Confirm configuration file was created] ***
ok: [172.31.17.140] => {
"changed": false,
"msg": "Application configuration exists with restricted permissions."
}
ok: [172.31.16.206] => {
"changed": false,
"msg": "Application configuration exists with restricted permissions."
}
PLAY RECAP ***
172.31.16.206 : ok=3 changed=1 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0
172.31.17.140 : ok=3 changed=1 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0
[root@ip-172-31-27-155 ansible]#
i-00b75b5c83d613bc0 (ansible-controller)
PublicIPs: 98.80.5.253 PrivateIPs: 172.31.27.155